Principios de la Robótica Autónoma

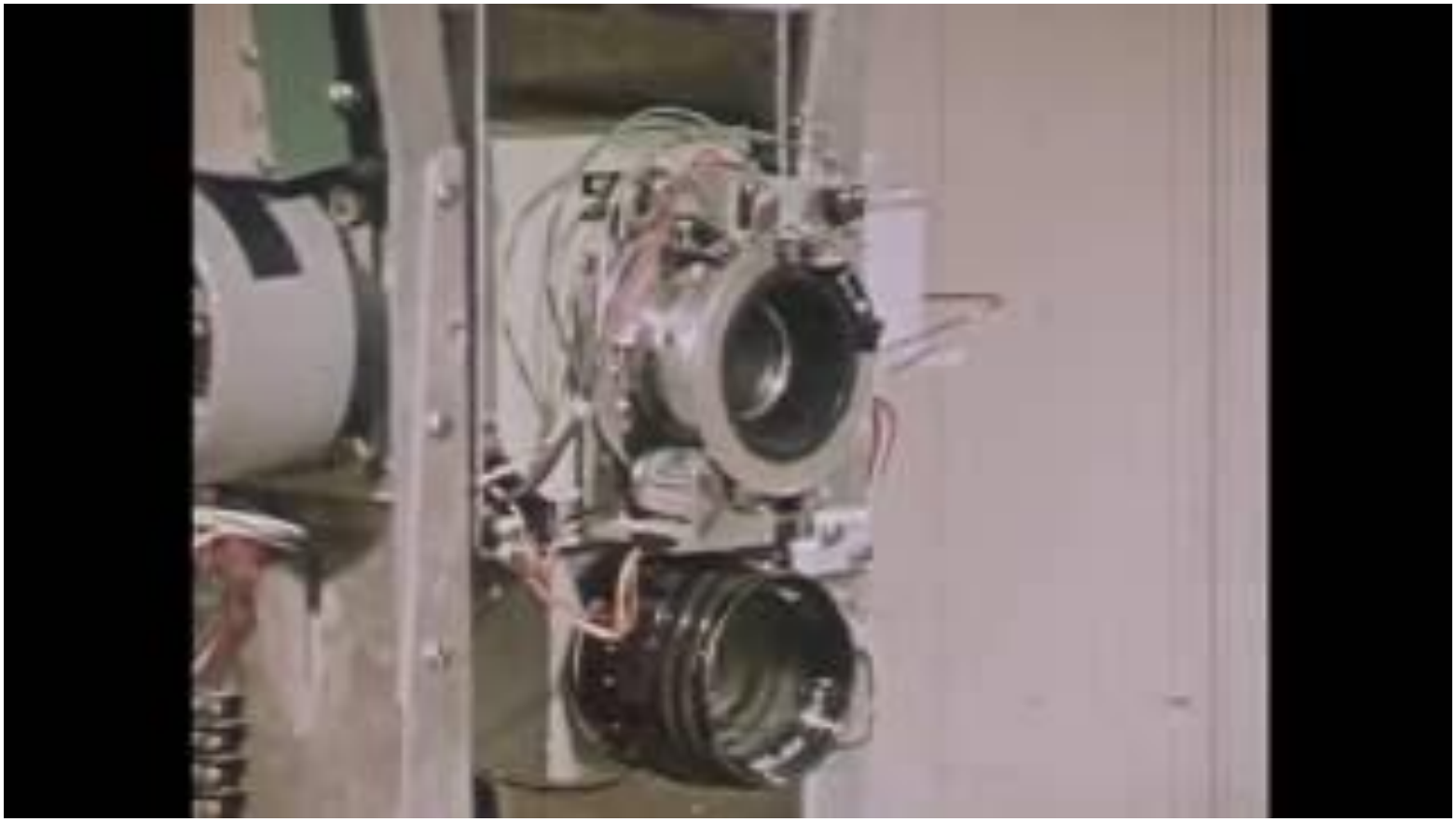
Paradigmas de control de robots

Ignacio Mas

Clásico / Paradigma Jerárquico

- Años 70
- Foco en razonamiento automático y representación de conocimiento
- STRIPS (Stanford Research Institute Problem Solver):
  - Modelo perfecto del mundo, asumiendo un mundo cerrado y limitado
- Tarea: encontrar cajas y moverlas a posiciones designadas

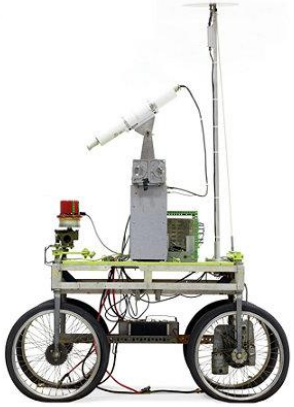
SRI STRIPS 1972



Stanford CART 1973

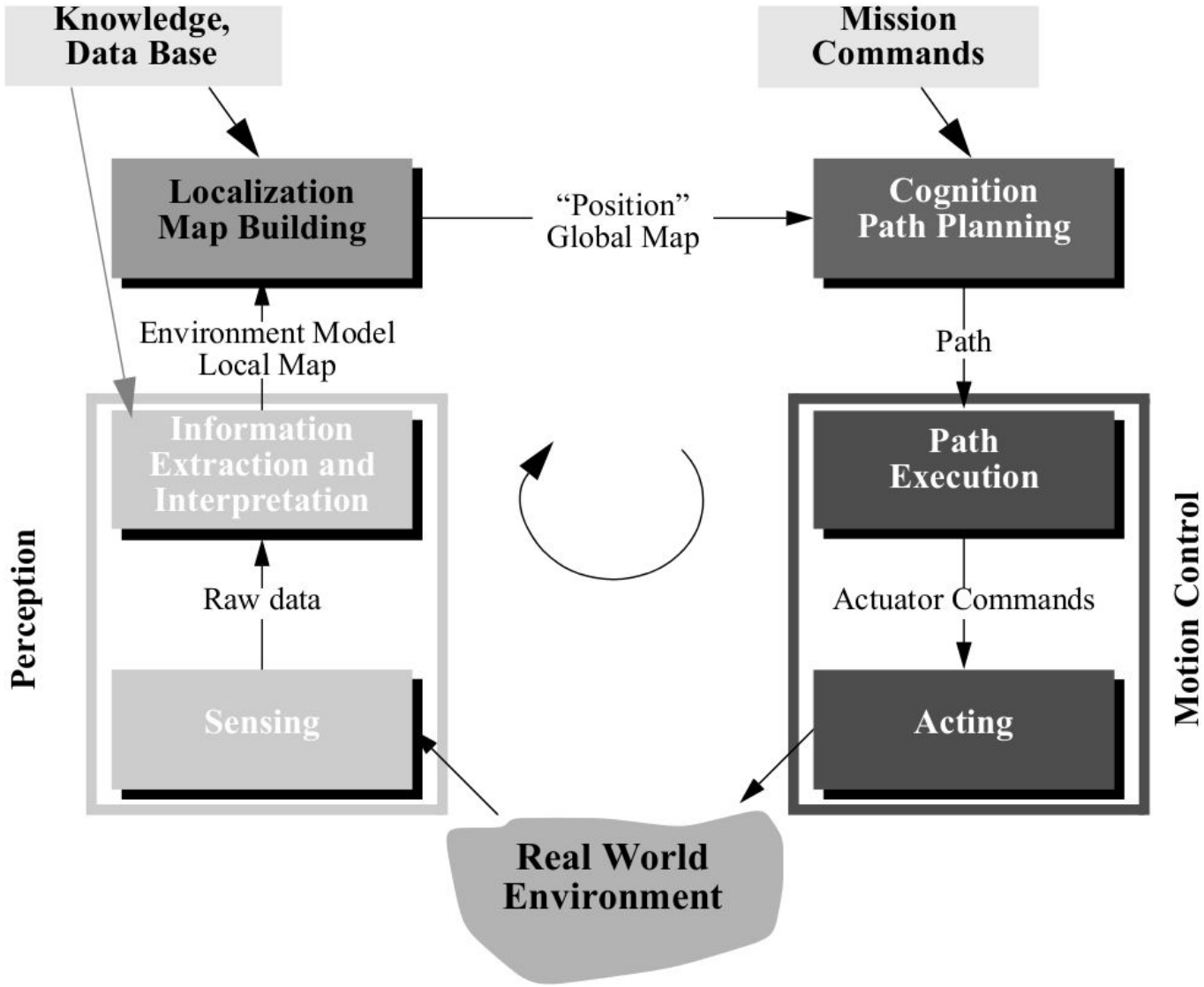
Stanford AI Laboratory / CMU (Moravec)

Paradigma clásico
Stanford Cart

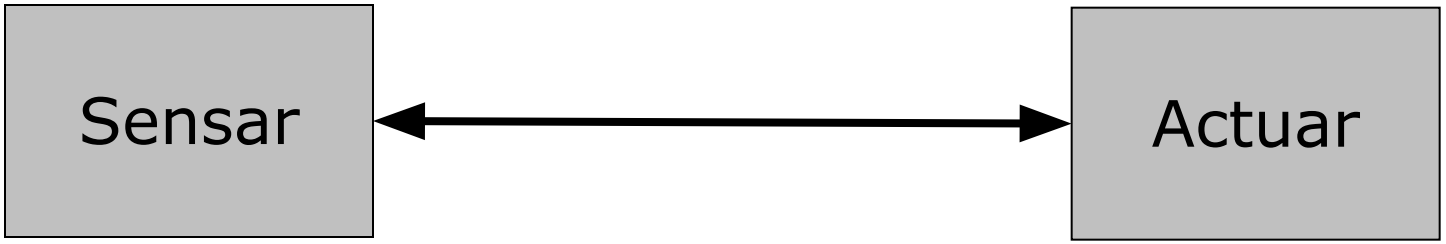


1. Tomar 9 imágenes del entorno, identificar puntos interesantes de 1 imagen y usar las otras imágenes para obtener estimaciones de profundidad.
2. Integrar la información con un modelo global del mundo.
3. Correlacionar las imágenes con imágenes previas para estimar el movimiento de robot.
4. Basándose en el movimiento deseado, el estimado, y el estado actual del entorno, determinar en qué dirección moverse.
5. Ejecutar el movimiento.

Paradigma clásico como descomposición funcional

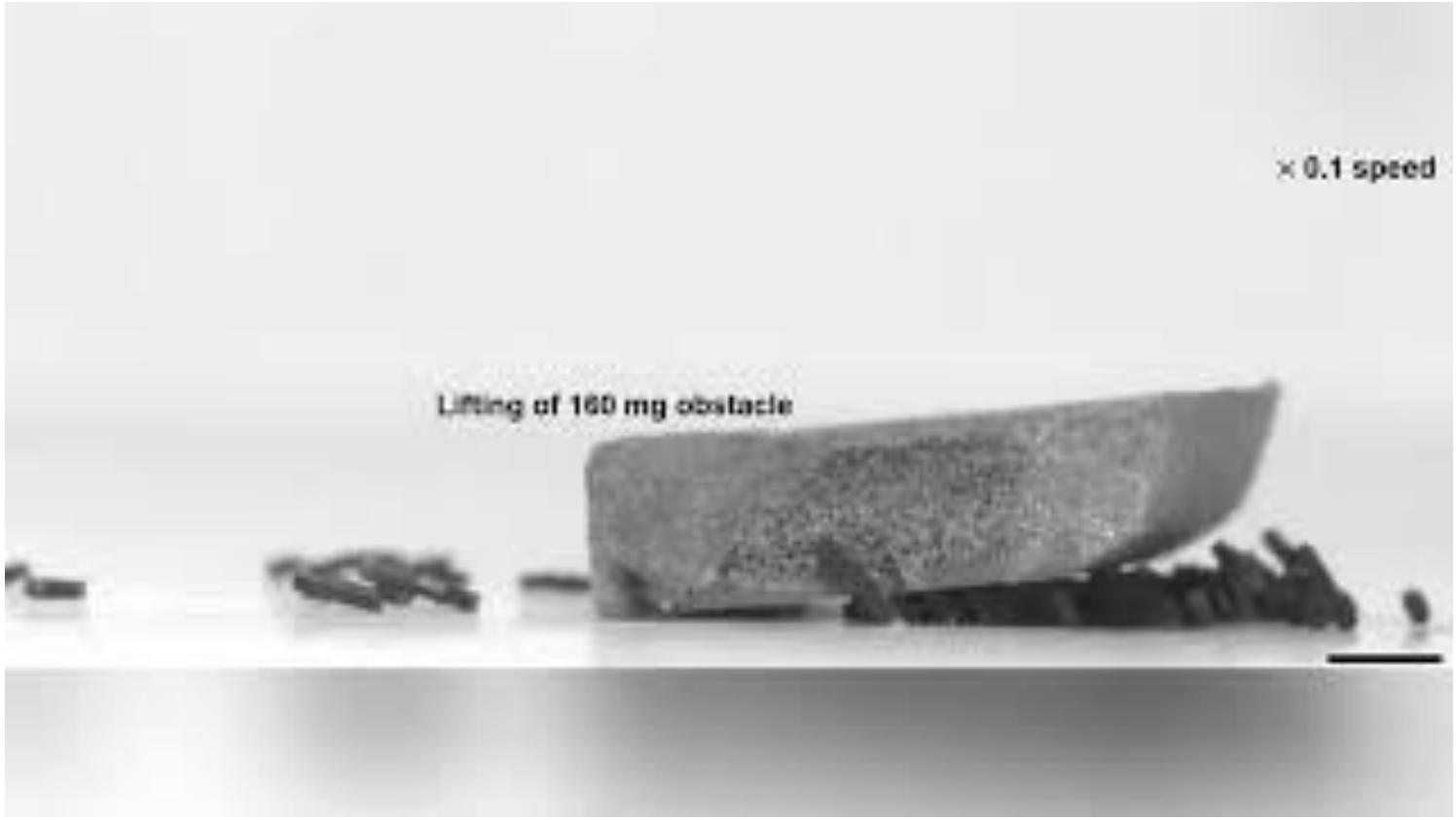


Paradigma basado en comportamientos (reactivo)

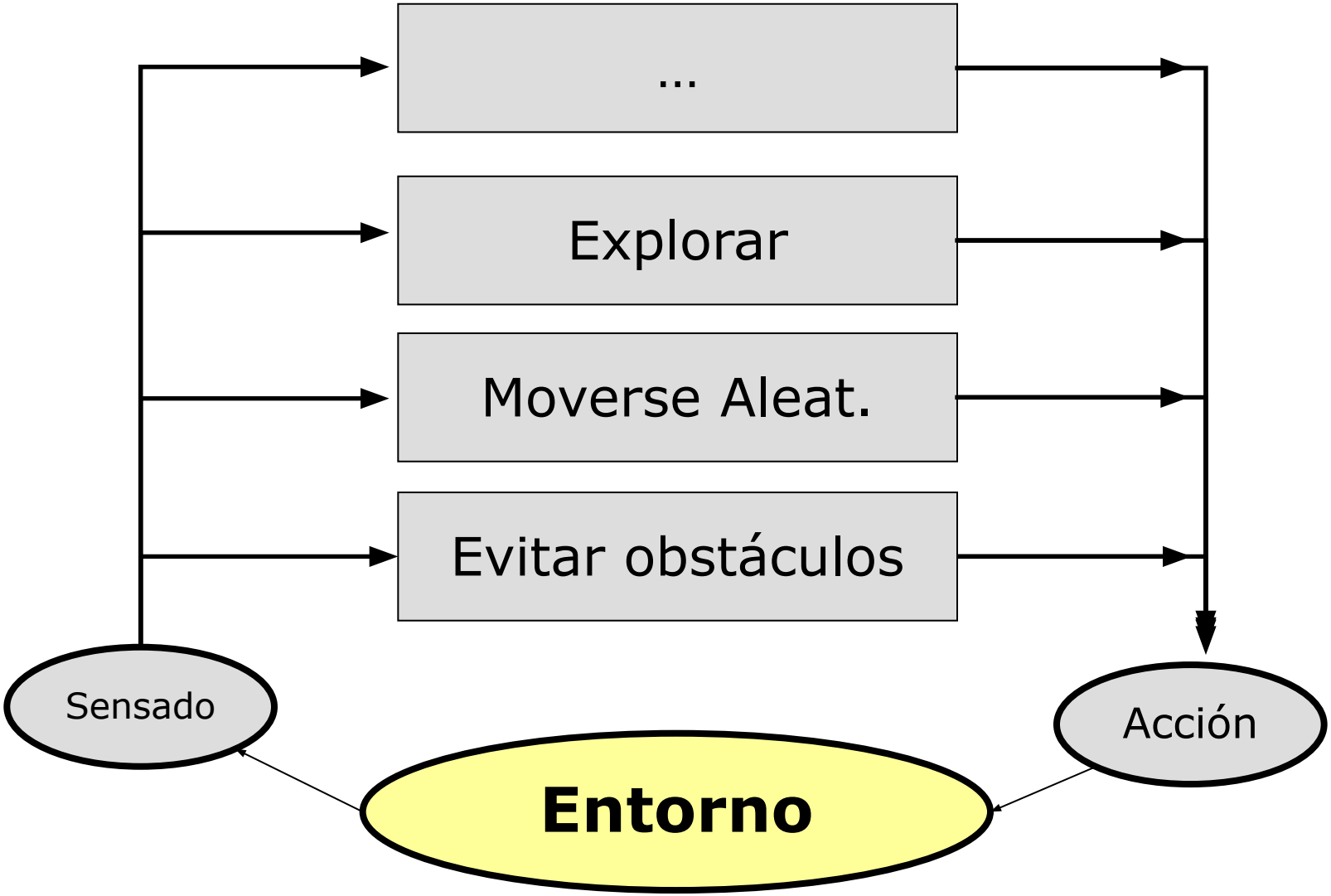


- No hay modelos:
  - “El mundo mismo es su mejor modelo”
- Inspirados en sistemas biológicos
- Comienzos exitosos, pero limitados

Swarms de micro-robots



Paradigma reactivo (descomposición vertical)



Características del Paradigma reactivo

- Agente Situado, el robot es parte integral de su entorno
- Sin memoria, controlado por lo que está sucediendo a su alrededor.
- Alto acoplamiento entre percepción y acción (mediante **comportamientos**).
- Solo existe un sensado local, específico al comportamiento (representación egocéntrica).

Comportamientos

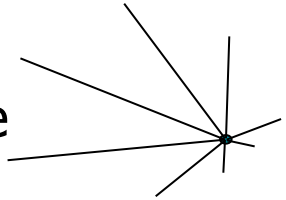
- ... son un mapeo directo de datos de sensores a un patrón de acciones motoras tendientes a realizar una tarea.
- ... son los bloques básicos de las acciones del robot, y el comportamiento general resultante del robot es emergente.
- ... permite un buen desarrollo de software debido a su modularidad.

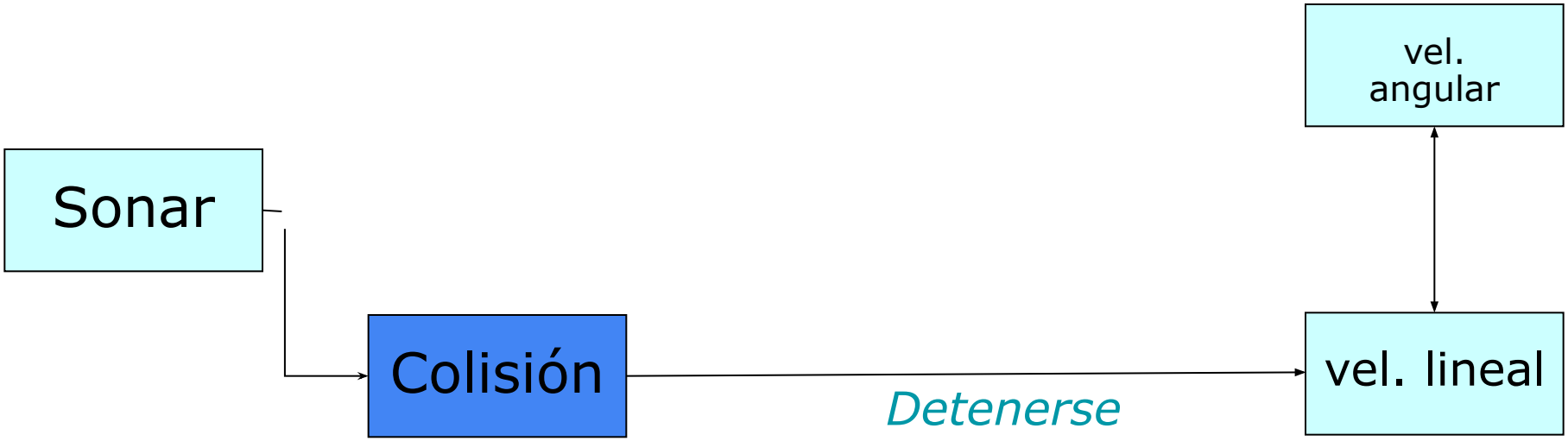
Arquitectura de Subsunción

- Introducida por Rodney Brooks ‘86.
- Los comportamientos son redes de nodos de sensado-actuación (augmented finite state machines AFSM).
- Los módulos se agrupan en capas de competencia.
- Una capa subsume capas inferiores.
- ¡No hay un estado interno!

Nivel 0: Colisión

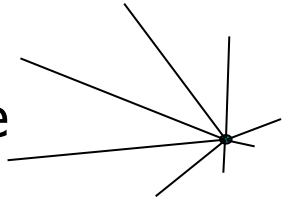
Gráfico polar de un sonar:

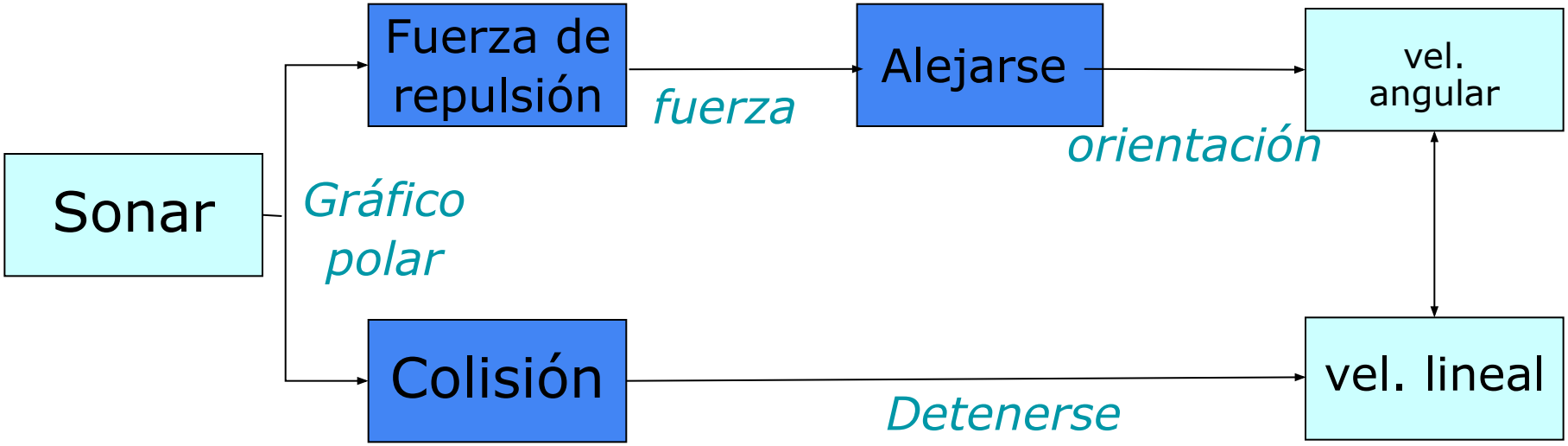




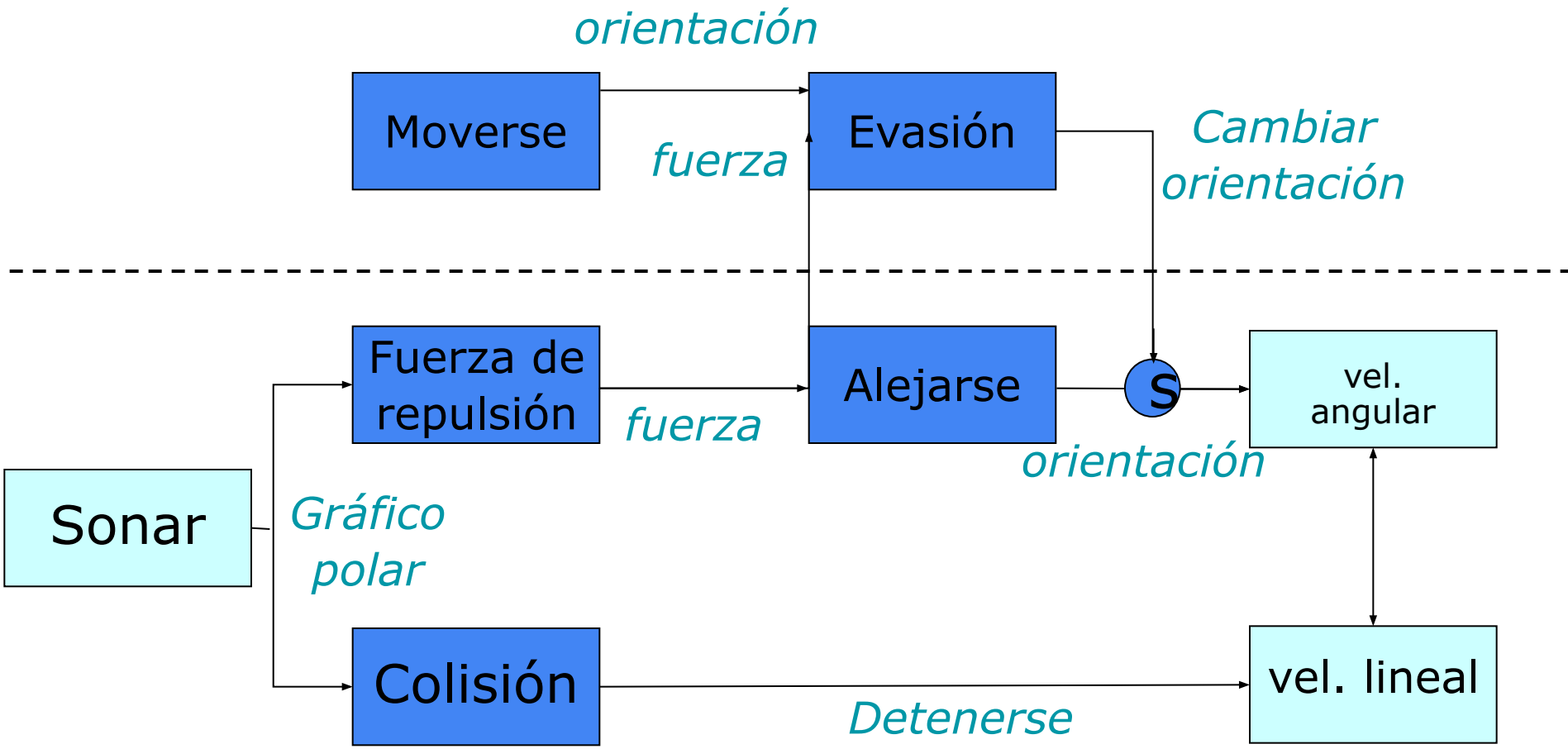
Nivel 0: Evasión

Gráfico polar de un sonar:

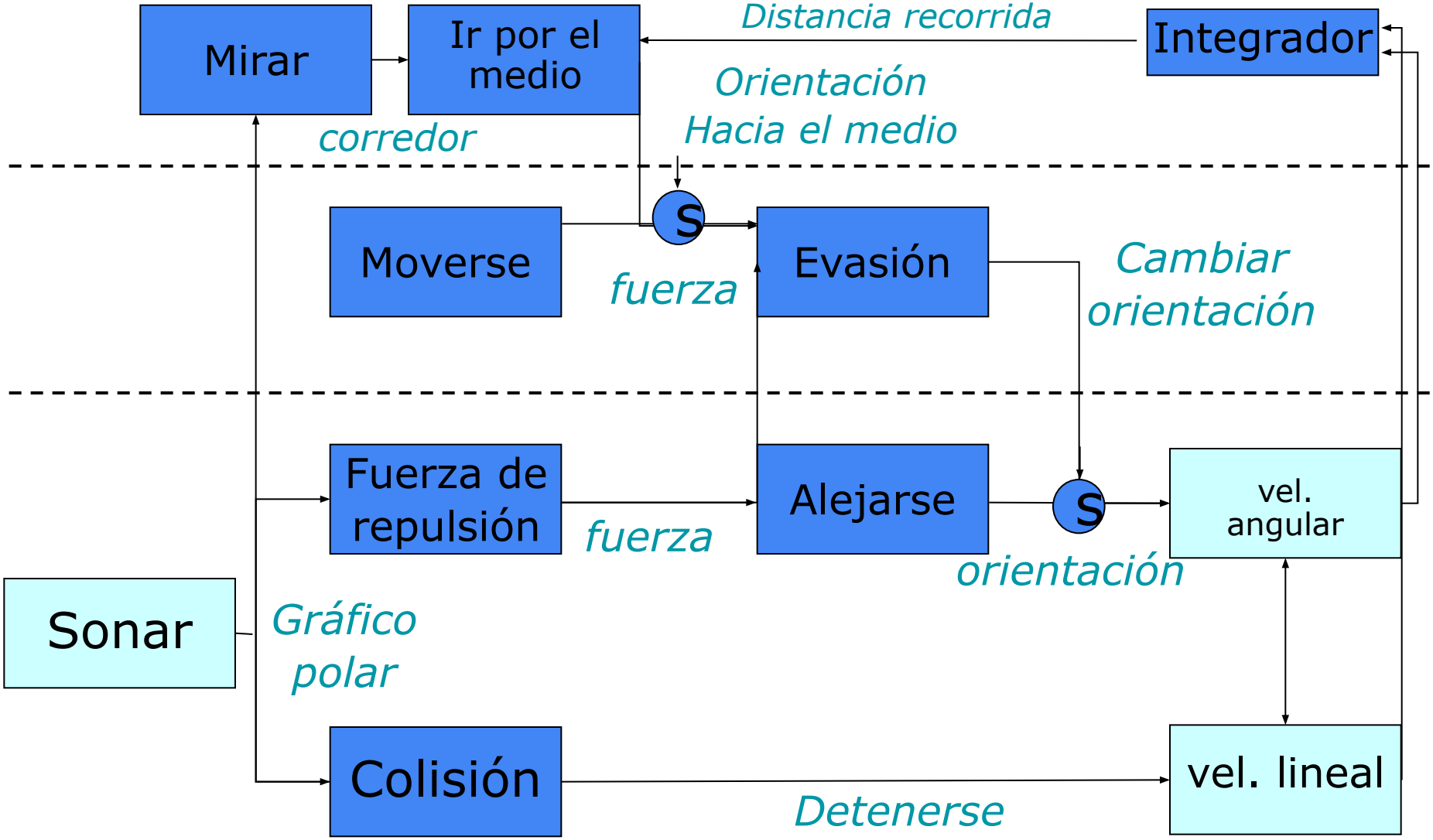




Nivel 1: Moverse aleatoriamente

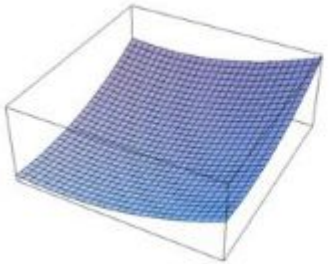


Nivel 2: Seguir un corredor

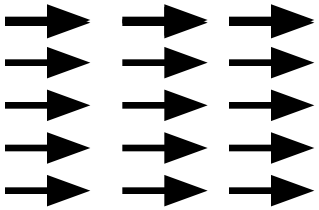


Métodos de campos potenciales

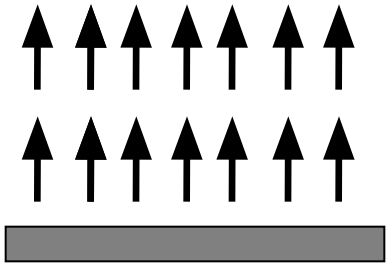
- Los robots son partículas que actúan bajo la influencia de un campo potencial
- Los robots siguen los gradientes de potencial
- El campo depende de obstáculos, dirección deseada y objetivos
- El campo resultante (vectorial) está dado por una suma de campos básicos (primitivos)
- La fuerza del campo varía con la distancia a los obstáculos y objetivos



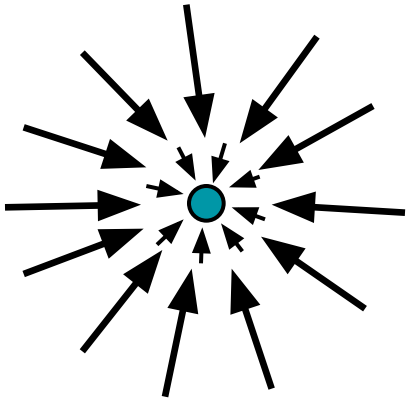
Primitivas de Campos Potenciales



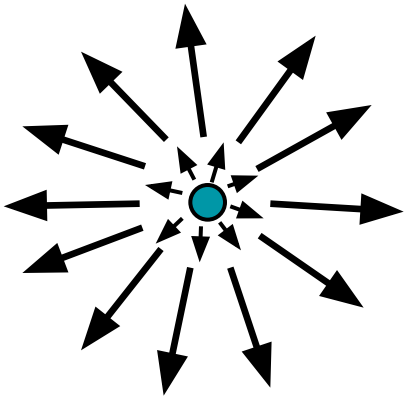
Uniforme



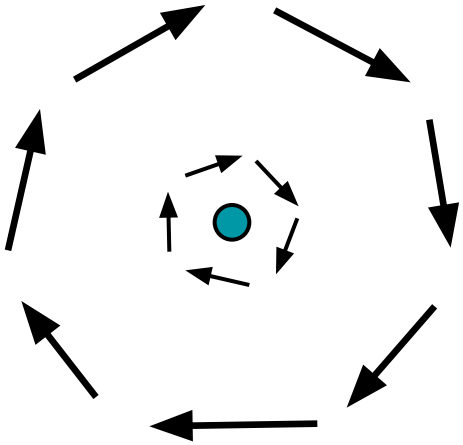
Perpendicular



Atractivo

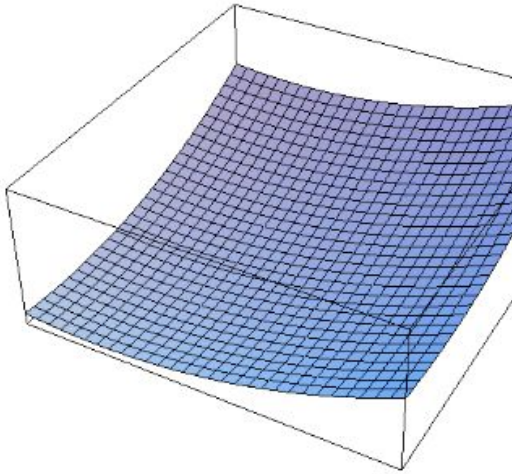


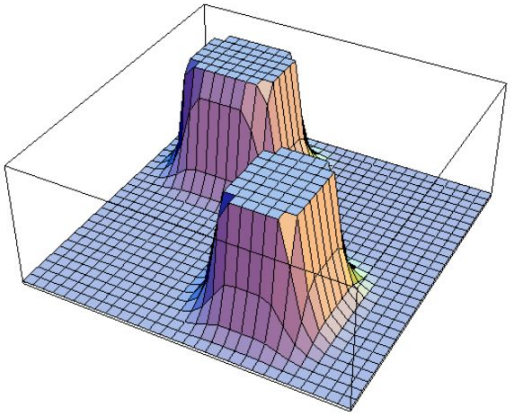
Repulsivo

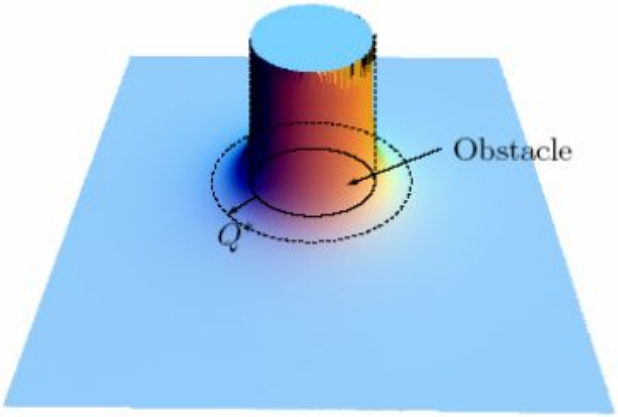


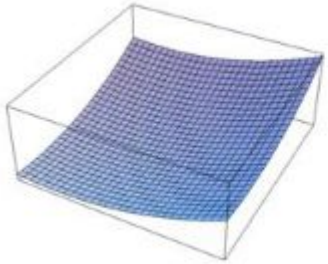
Tangencial

Primitivas de Campos Potenciales

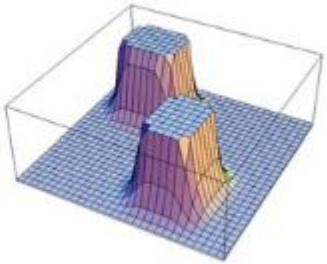




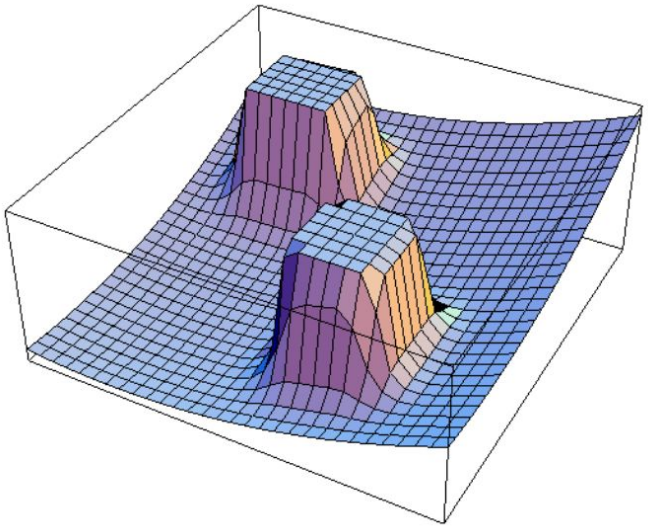




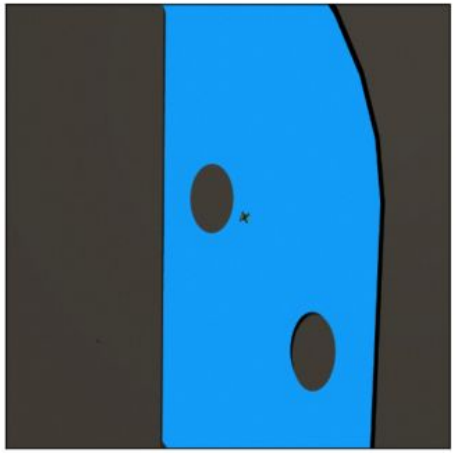
+



=

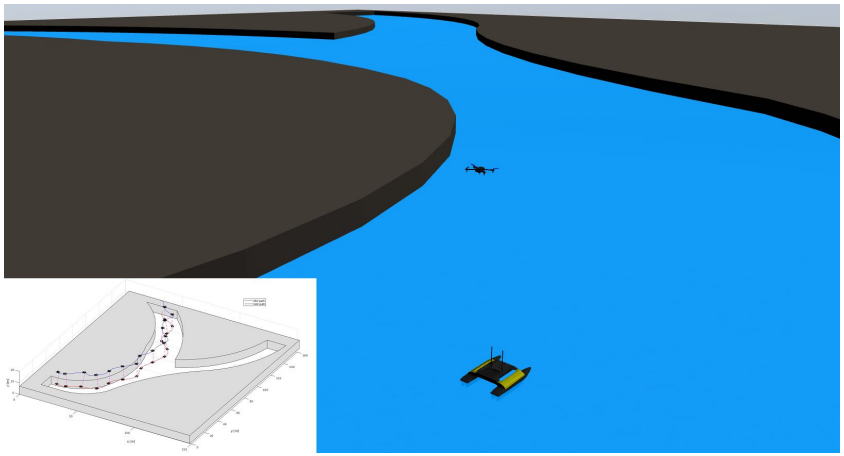


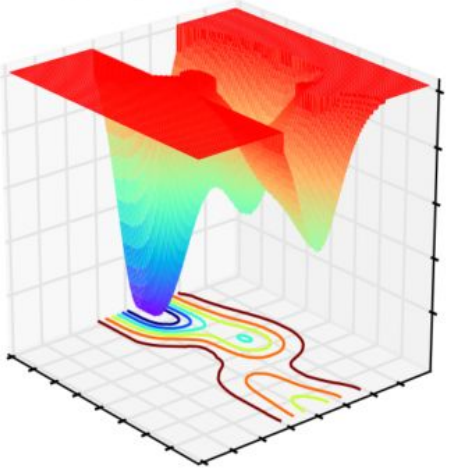
Ejemplo de Campos Potenciales



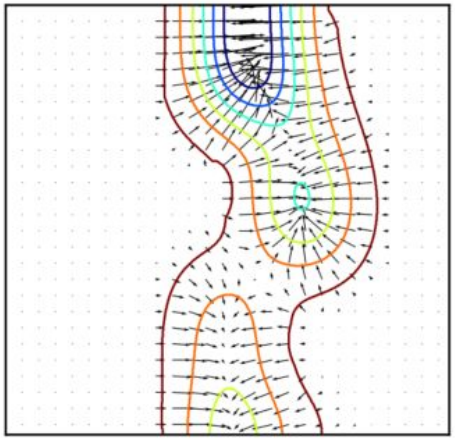
(a) Captura de la cámara

(b) Potencial en escala de grises

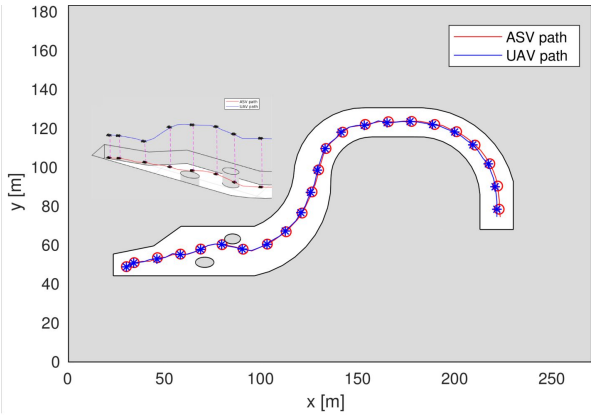




(c) Potencial generado



(d) Campo vectorial



Seguir un corredor usando Campos Potenciales

- Nivel 0 (evasión de obstáculos)

  fuerzas repulsivas debido a obstáculos detectados.

- Nivel 1 (moverse)

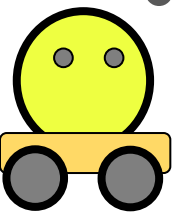
  campo uniforme.

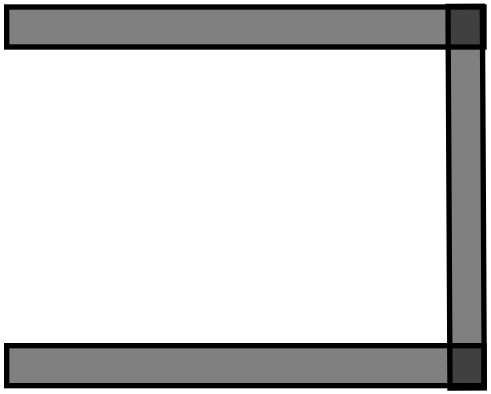
- Nivel 2 (seguimiento de un corredor)

  reemplaza el campo uniforme con 3 campos (dos perpendiculares, uno uniforme).

Características de campos potenciales

- Problema de mínimos locales

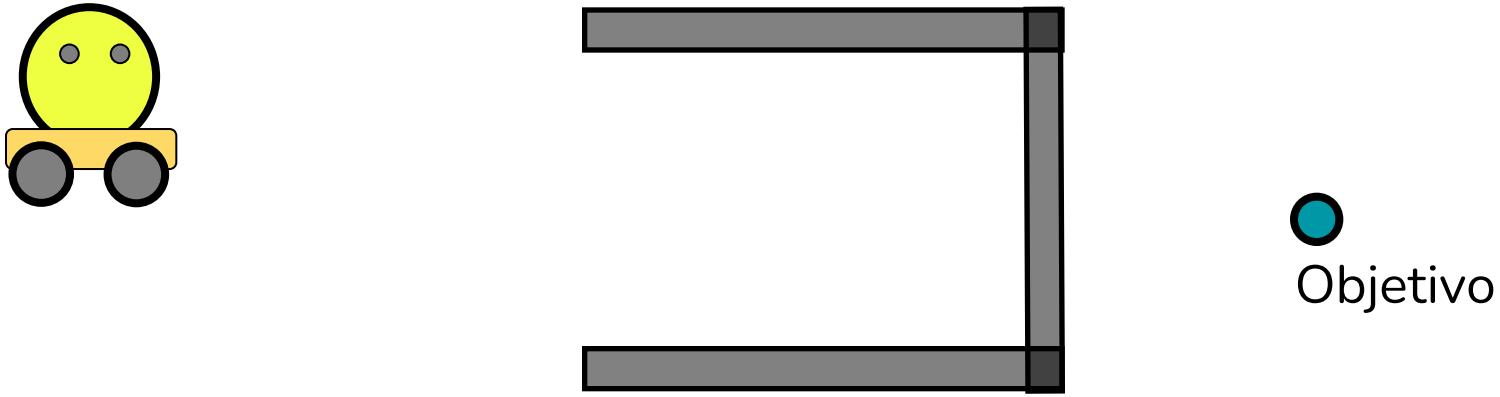




Objetivo

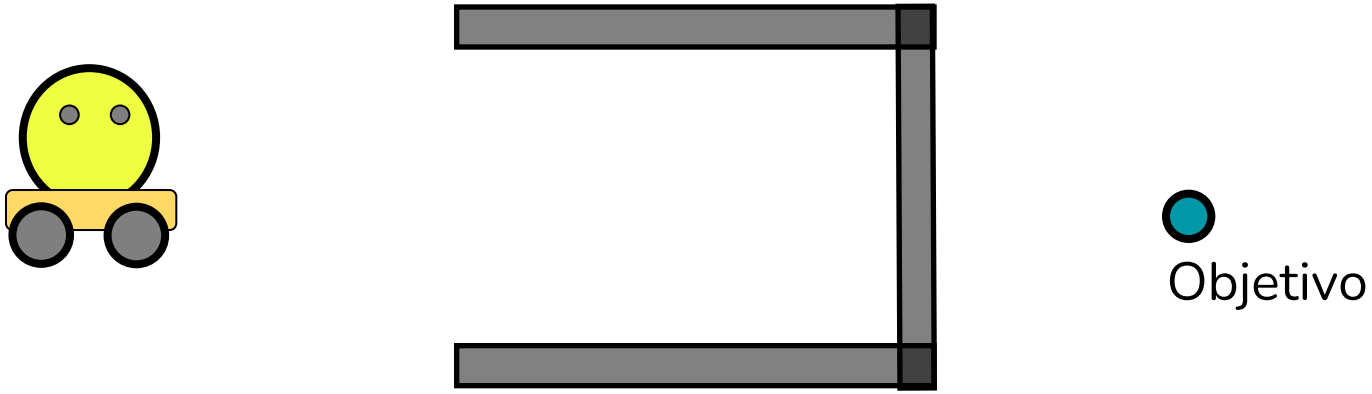
Características de campos potenciales

- Problema de mínimos locales



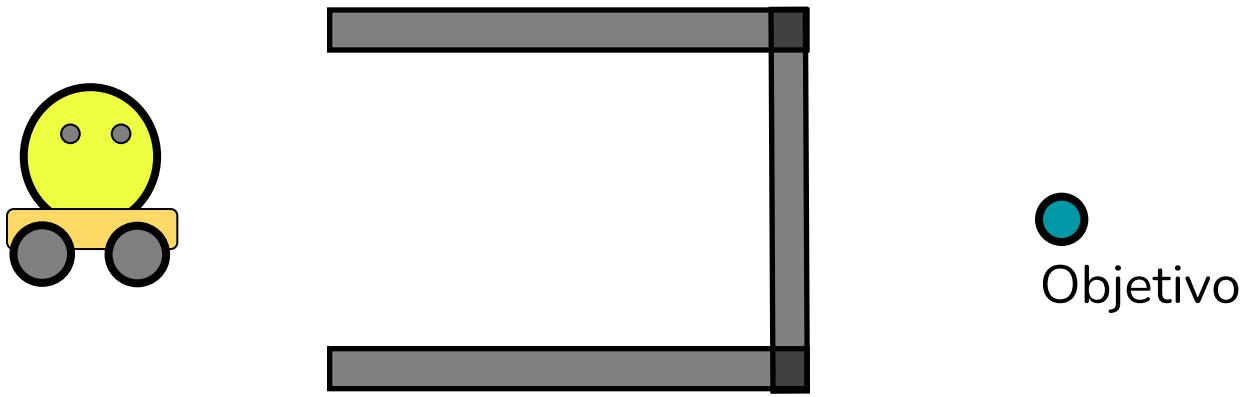
Características de campos potenciales

- Problema de mínimos locales



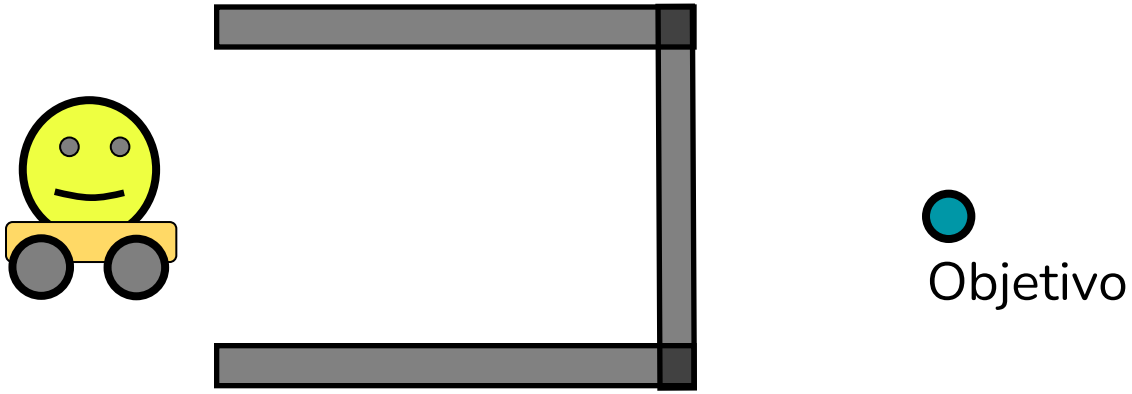
Características de campos potenciales

- Problema de mínimos locales



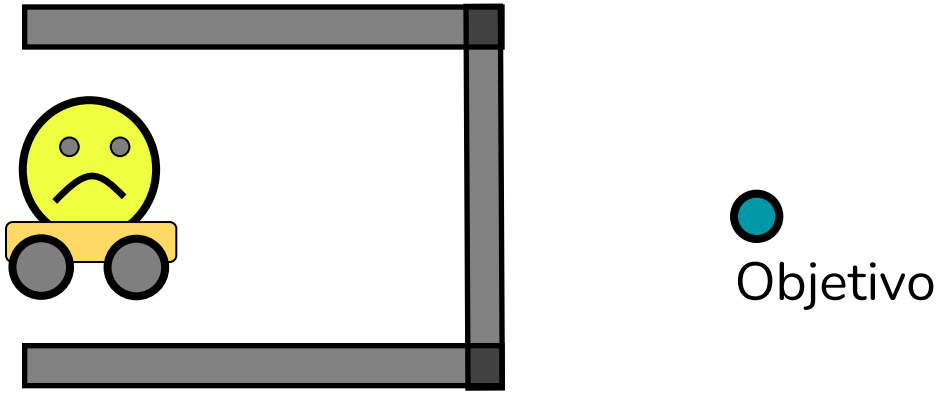
Características de campos potenciales

- Problema de mínimos locales



Características de campos potenciales

- Problema de mínimos locales

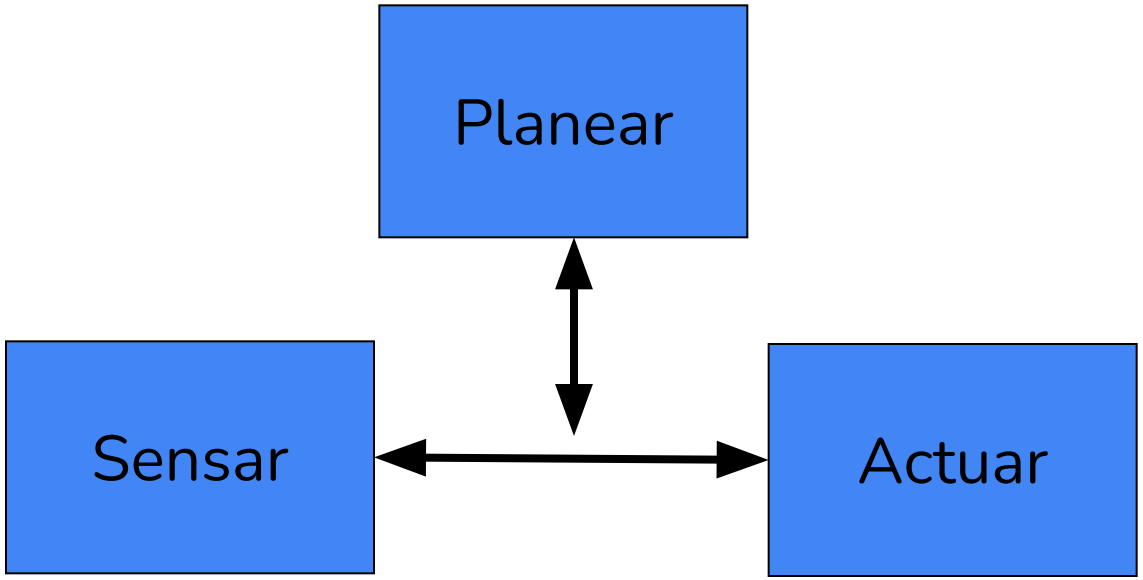


- Backtracking
- Movimiento aleatorio para escapar de mínimos locales
- Planeamiento (ejemplo: seguimiento de paredes)
- Aumentar potencial de regiones visitadas

Características de campos potenciales

- No hay **prioridad** entre niveles
- Fácil de **visualizar**
- Fácil **combinación** entre campos
- Necesidad de alta tasa de actualización
- Muy dependiente del **sintonizado** de parámetros

Paradigma Híbrido Deliberativo/Reactivo



- Combina ventajas de los paradigmas anteriores
  - Un modelo el mundo para planeamiento
  - Control reactivo

Discusión

- ¿Si uno quiere un robot que pueda hacer tareas de navegación, qué método elegiría?

- ¿Cuáles son los beneficios y limitaciones del paradigma basado en comportamiento?

- ¿Cuáles son las desventajas de la arquitectura de subsunción?

Arquitecturas de Software: ROS

- ROS (Robot Operating System) nació en 2006 en Stanford, con el objetivo de evitar que los investigadores **reinventaran una y otra vez la infraestructura básica** de robótica (drivers, comunicación, etc.) y así enfocarse en desarrollar inteligencia robótica.

- Aunque existían sistemas similares (Player/Stage, URBI, YARP), ROS se diferenció por ser **open source** y, sobre todo, por construir una gran **comunidad**.

- En 2008, crearon la **start-up Willow Garage**, donde ROS creció exponencialmente, integró el simulador Gazebo y lanzó el Turtlebot como plataforma asequible.

- Tras el cierre de Willow Garage en 2013, el desarrollo pasó a la Open Source Robotics Foundation (luego Open Robotics), que continuó con nuevas distribuciones y sentó las bases de **ROS 2** para superar limitaciones de ROS 1 (seguridad, tiempo real, fiabilidad). ROS 2 ha atraído el interés de grandes empresas como Amazon, Google y Microsoft.

- Hoy, ROS es el **estándar de facto en robótica**, con una comunidad global y soporte industrial

Arquitecturas de Software

ROS 1

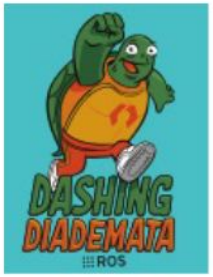
2014 - 2019

2016 - 2021

2018 - 2023

2020 - 2025

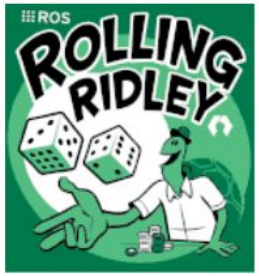
ROS 2



2019 - 2021

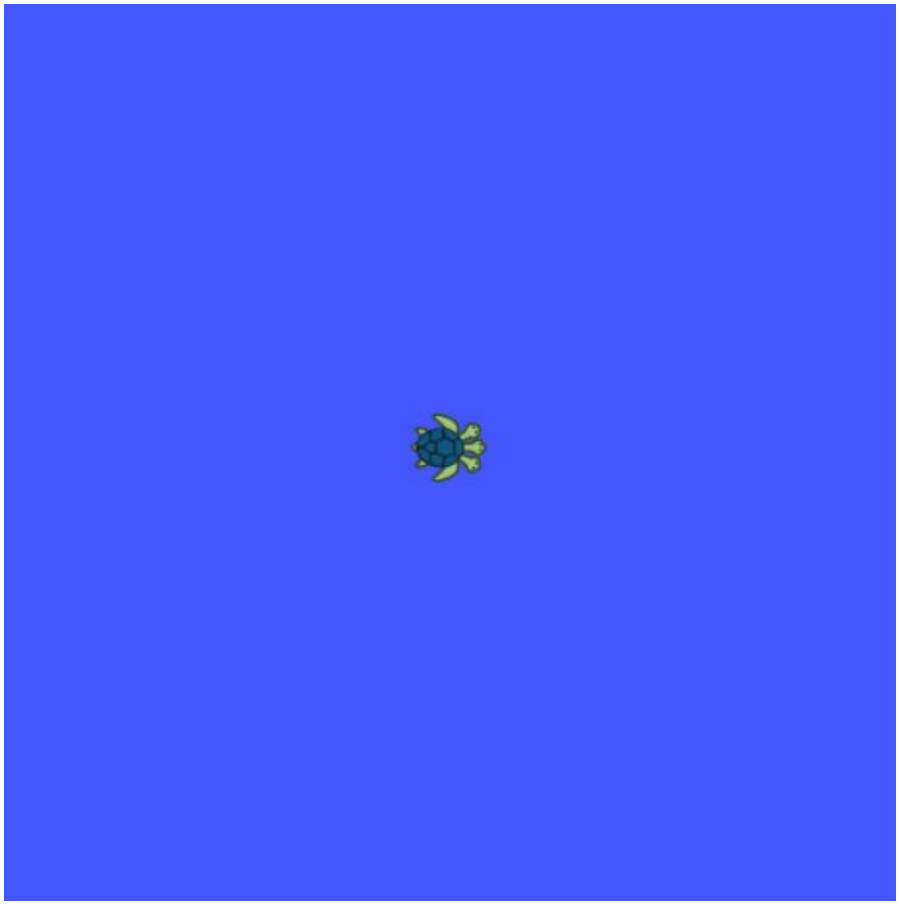
2020 - 2023

2022 - 2027



2017 - Present

ROS - TurtleSim



Por qué ROS es popular

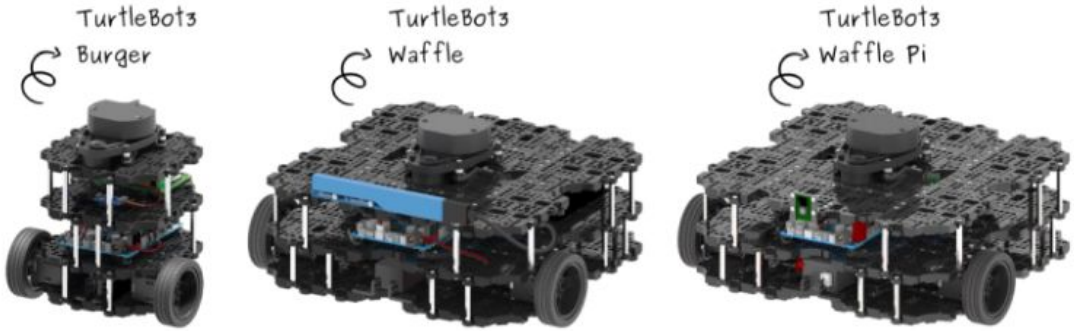
- No reinventar la rueda!
- Simplificación de la arquitectura
- Estandarización
- Herramientas comunes

Por qué ROS es popular

- Comunidad y desarrollo
  - Drivers de hardware

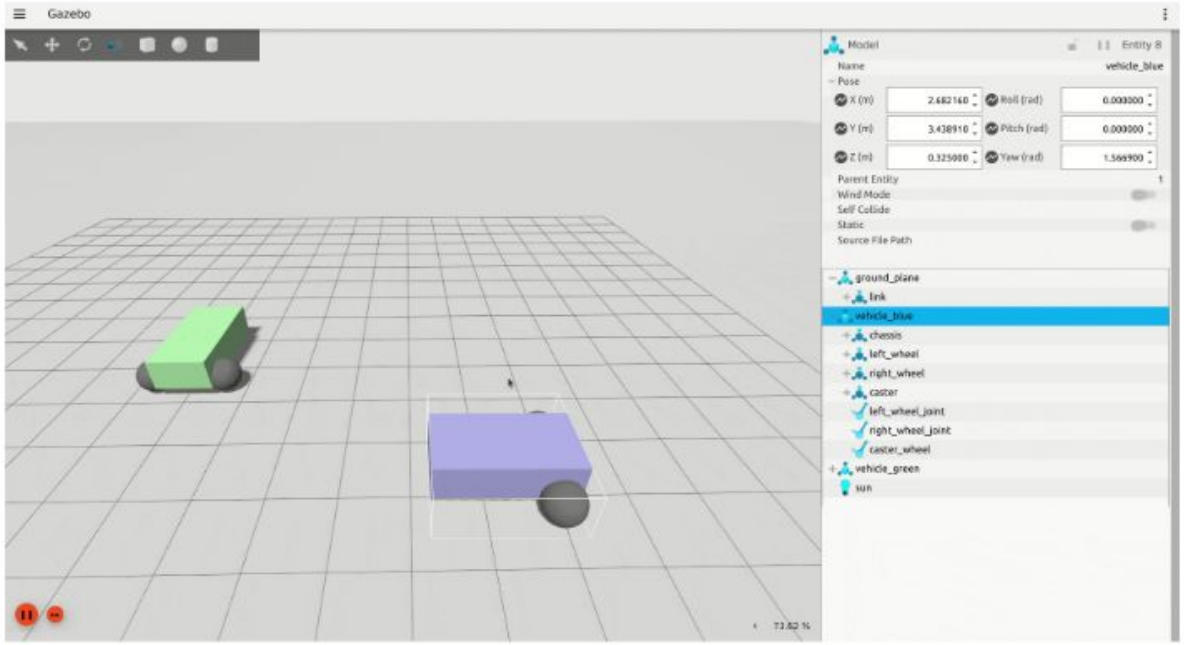
Crazyflie

Joysticks



Por qué ROS es popular

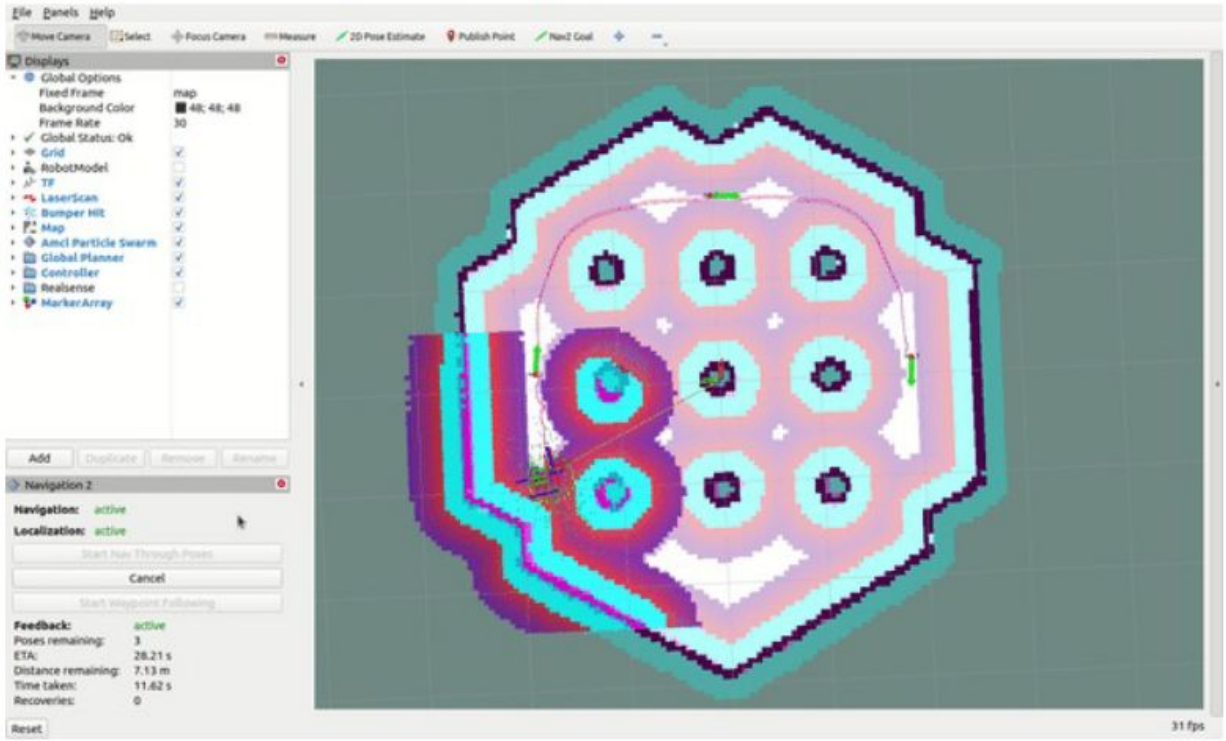
- Comunidad y desarrollo
  - Drivers de hardware
  - Software



Gazebo Sim

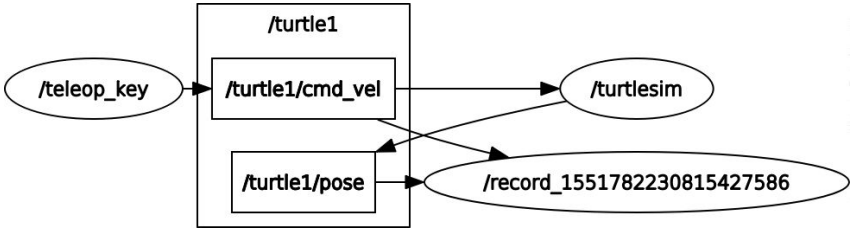
Por qué ROS es popular

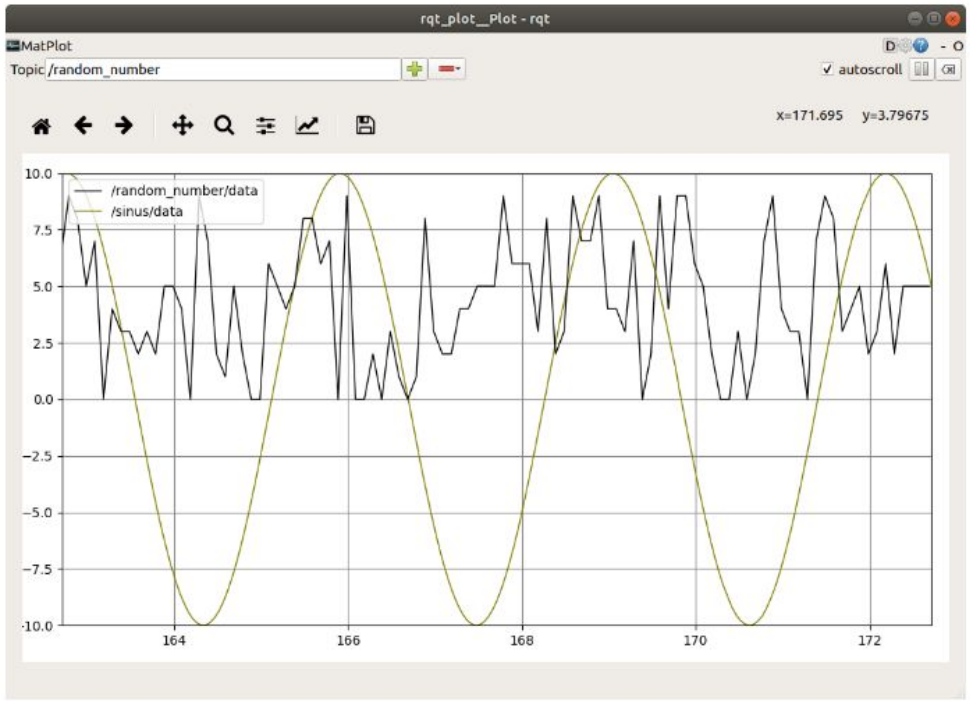
- Desarrollo de herramientas
  - Visualización

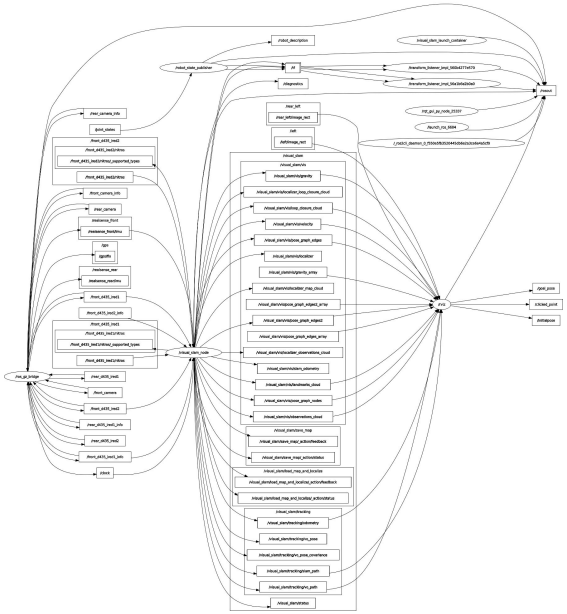


Por qué ROS es popular

- Desarrollo de herramientas
  - Visualización
  - Debugging

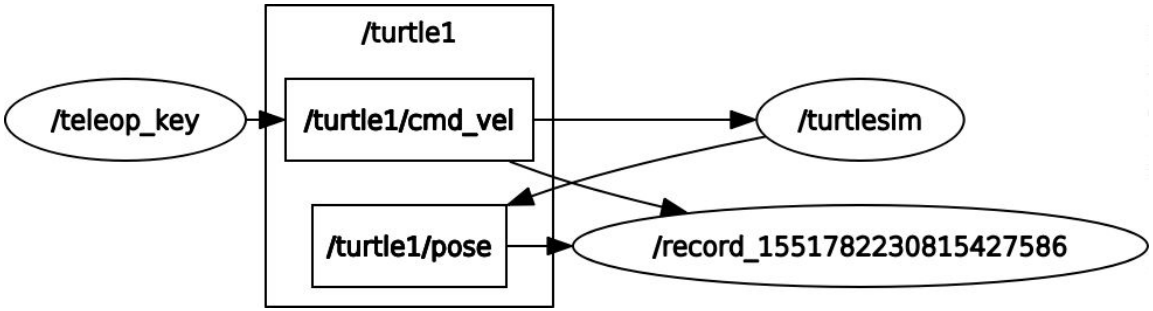






Por qué ROS es popular

- Desarrollo asincrónico de software
  - Modelo de nodos
  - Comunicación a través mensajes
  - Subscripción a tópicos / publicación en tópicos



ROS Ejemplo

ROBOTIC SIMULATOR
IN UNREAL ENGINE
HUMBLE
HAWKSBILL
UNREAL
ENGINE
UNREAL ENGINE 5 WITH ROS2